

L10 — Spring Boot REST API with JPA/ORM

Question-1: Steps to set up Spring Boot + REST + JPA (Maven deps: spring-boot-starter-web, spring-boot-starter-data-jpa, MySQL connector) and role of embedded Tomcat.

Answer:

Step 1: Create a Spring Boot Maven Project

Required dependencies:

```
<dependencies>
```

```
  <!-- Spring Web / REST API -->
```

```
  <dependency>
```

```
    <groupId>org.springframework.boot</groupId>
```

```
    <artifactId>spring-boot-starter-web</artifactId>
```

```
  </dependency>
```

```
  <!-- Spring Data JPA -->
```

```
  <dependency>
```

```
    <groupId>org.springframework.boot</groupId>
```

```
    <artifactId>spring-boot-starter-data-jpa</artifactId>
```

```
  </dependency>
```

```
  <!-- MySQL Connector -->
```

```
  <dependency>
```

```
    <groupId>com.mysql</groupId>
```

```
    <artifactId>mysql-connector-j</artifactId>
```

```
    <scope>runtime</scope>
```

```
  </dependency>
```

```
</dependencies>
```

Step 2: Configure MySQL

```
application.properties  
  
spring.datasource.url=jdbc:mysql://localhost:3306/studentdb  
  
spring.datasource.username=root  
  
spring.datasource.password=1234  
  
spring.jpa.hibernate.ddl-auto=update  
  
spring.jpa.show-sql=true  
  
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQLDialect
```

Step 3: Create Entity

```
@Entity  
  
public class Student {  
  
    @Id  
  
    @GeneratedValue(strategy = GenerationType.IDENTITY)  
  
    private Long id;  
  
    private String name;  
  
    private double cgpa;  
  
    // Getters and Setters  
  
}
```

Step 4: Create Repository

```
public interface StudentRepository  
  
    extends JpaRepository<Student, Long> {  
  
}
```

Step 5: Create REST Controller

```
@RestController  
  
@RequestMapping("/students")
```

```

public class StudentController {

    @Autowired

    private StudentRepository repository;

    @GetMapping

    public List<Student> getStudents() {

        return repository.findAll(); }

    @PostMapping

    public Student addStudent(@RequestBody Student student) {

        return repository.save(student);

    } }

```

Role of Embedded Tomcat

Spring Boot includes **embedded Tomcat** by default when using spring-boot-starter-web.

It:

- Runs the Spring Boot application directly.
- Provides the web server for REST APIs.
- Usually runs on port **8080**.
- Removes the need to manually install/deploy a separate Tomcat server.

For example:

http://localhost:8080/students

Practical-1: JPA Student entity (id, name, cgpa) + StudentRepository extends JpaRepository.

Answer:

Student.java

```
package com.example.demo;
```

```
import jakarta.persistence.Entity;

import jakarta.persistence.GeneratedValue;

import jakarta.persistence.GenerationType;

import jakarta.persistence.Id;

@Entity

public class Student {

    @Id

    @GeneratedValue(strategy = GenerationType.IDENTITY)

    private Long id;

    private String name;

    private double cgpa;

    public Student() { }

    public Long getId() {

        return id; }

    public void setId(Long id) {

        this.id = id; }

    public String getName() {

        return name; }

    public void setName(String name) {

        this.name = name; }

    public double getCgpa() {

        return cgpa; }

    public void setCgpa(double cgpa) {

        this.cgpa = cgpa;

    } }
```

StudentRepository.java

```
package com.example.demo;

import org.springframework.data.jpa.repository.JpaRepository;

public interface StudentRepository

    extends JpaRepository<Student, Long> {

}
```

Explanation

- @Entity → Makes Student a database entity.
- @Id → Defines the primary key.
- @GeneratedValue → Automatically generates ID.
- JpaRepository → Provides CRUD operations such as:
 - save()
 - findAll()
 - findById()
 - deleteById()

Practical-2: @RestController with GET /students and POST /students endpoints using the repository.

Answer:

StudentController.java

```
package com.example.demo;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController

@RequestMapping("/students")
```

```
public class StudentController {  
    @Autowired  
    private StudentRepository repository;  
    // GET /students  
    @GetMapping  
    public List<Student> getStudents() {  
        return repository.findAll(); }  
    // POST /students  
    @PostMapping  
    public Student addStudent(@RequestBody Student student) {  
        return repository.save(student);  
    }  
}
```

Test GET

GET <http://localhost:8080/students>

Example response:

```
[  
  {  
    "id": 1,  
    "name": "Alif",  
    "cgpa": 3.75  
  },  
  {  
    "id": 2,  
    "name": "Rahim",  
    "cgpa": 3.50  
  }  
]
```

```
]
```

Test POST

POST http://localhost:8080/students

JSON body:

```
{  
  "name": "Alif",  
  "cgpa": 3.75  
}
```

The repository.save(student) method stores the student in MySQL.

L11 — Servlet CRUD — District Quiz Game

Question-1: Design DB schema for quiz on Crops / Geography / Academic Institutions of your district + Player/Score table. Justify design.

Answer:

For a district quiz game, we can use three main tables:

1. Questions Table

```
CREATE TABLE Questions (  
  question_id INT PRIMARY KEY AUTO_INCREMENT,  
  question_text VARCHAR(255),  
  option_a VARCHAR(100),  
  option_b VARCHAR(100),  
  option_c VARCHAR(100),  
  option_d VARCHAR(100),  
  correct_option CHAR(1),  
  category VARCHAR(50)  
);
```

Categories can be:

Crop

Geography

Academic Institution

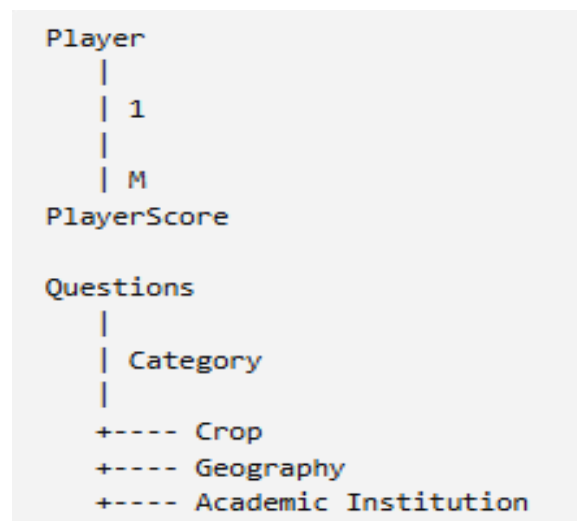
2. Player Table

```
CREATE TABLE Player (  
    player_id INT PRIMARY KEY AUTO_INCREMENT,  
    player_name VARCHAR(100)  
);
```

3. PlayerScore Table

```
CREATE TABLE PlayerScore (  
    score_id INT PRIMARY KEY AUTO_INCREMENT,  
    player_id INT,  
    final_score INT,  
    FOREIGN KEY (player_id)  
        REFERENCES Player(player_id)  
);
```

Relationship



Justification

This design is good because:

1. Questions are separated from player information.
2. A player can have multiple scores.
3. category allows different types of quiz questions.
4. Primary keys uniquely identify records.
5. Foreign key maintains relationship between Player and PlayerScore.
6. The design reduces unnecessary data duplication.

Practical-1: Servlet method: save Player name + final score into PlayerScore via JDBC/PreparedStatement.

Answer:

PlayerScoreServlet.java

```
import jakarta.servlet.*;
import jakarta.servlet.http.*;
import java.io.*;
import java.sql.*;

public class PlayerScoreServlet extends HttpServlet {

    private static final String URL =

        "jdbc:mysql://localhost:3306/quizapp";

    private static final String USER = "root";

    private static final String PASSWORD = "1234";

    protected void doPost(HttpServletRequest request,

        HttpServletResponse response)

        throws ServletException, IOException {

        String playerName = request.getParameter("playerName");
```

```

int score = Integer.parseInt(
    request.getParameter("score"));
String sql = ""
    INSERT INTO PlayerScore(player_name, final_score)
    VALUES (?, ?)
    "",
try {
    Class.forName("com.mysql.cj.jdbc.Driver");
    Connection con =
        DriverManager.getConnection(
            URL, USER, PASSWORD);
    PreparedStatement ps =
        con.prepareStatement(sql);
    ps.setString(1, playerName);
    ps.setInt(2, score);
    ps.executeUpdate();
    ps.close();
    con.close();
    response.getWriter().println(
        "Score saved successfully!");
} catch (Exception e) {
    e.printStackTrace();
} } }

```

PreparedStatement is preferred because:

- It prevents SQL Injection.
- It handles parameters safely.

- It improves readability.
- It can improve performance for repeated queries.

If using the normalized Player & PlayerScore design

You can first insert the player:

String sql1 =

```
"INSERT INTO Player(player_name) VALUES(?);"
```

Then insert the score using the generated player_id:

String sql2 =

```
"INSERT INTO PlayerScore(player_id, final_score) VALUES(?, ?);"
```

Practical-2: Quiz logic: ≥ 3 MCQs (crop, geography, institution) as objects; present, check answer, increment score.

Answer:

Suppose we have three MCQs:

1. Crop
2. Geography
3. Academic Institution

Question Class

```
class Question {
    String question;
    String[] options;
    int correctAnswer;
    Question(String question,
              String[] options,
              int correctAnswer) {
```

```

    this.question = question;

    this.options = options;

    this.correctAnswer = correctAnswer;
}

boolean checkAnswer(int answer) {

    return answer == correctAnswer;

}}

```

Quiz Logic

```

import java.util.*;

public class QuizGame {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        List<Question> questions = new ArrayList<>();

        questions.add(new Question(

            "Which crop is commonly grown in Lalmonirhat?",

            new String[]{"Tea", "Rice", "Cocoa", "Coffee"},

            2

        ));

        questions.add(new Question(

            "Lalmonirhat belongs to which division?",

            new String[]{"Dhaka", "Rangpur", "Sylhet", "Khulna"},

            2

        ));

        questions.add(new Question(

            "Which institution is located in Tangail?",

            new String[]{"MBSTU", "BUET", "DU", "RU"},

```

1

```
));  
int score = 0;  
for (Question q : questions) {  
    System.out.println("\n" + q.question);  
    for (int i = 0; i < q.options.length; i++) {  
        System.out.println(  
            (i + 1) + ". " + q.options[i]);    }  
    System.out.print("Enter answer: ");  
    int answer = sc.nextInt();  
    if (q.checkAnswer(answer)) {  
        System.out.println("Correct!");  
        score++;  
    } else {  
        System.out.println("Wrong!");  
    } }  
System.out.println("\nFinal Score = "  
    + score + "/" + questions.size());  
sc.close();  
} }
```

L12 — GoF Design Patterns

Question-1: List all 5 GoF Creational patterns (Singleton, Factory Method, Abstract Factory, Builder, Prototype) with one-line purpose each.

Answer:

Pattern	Purpose
Singleton	Ensures a class has only one object/instance and provides a global access point.
Factory Method	Creates objects without directly specifying the exact class to instantiate.
Abstract Factory	Creates families of related objects without specifying their concrete classes.
Builder	Constructs a complex object step-by-step and allows different representations.
Prototype	Creates new objects by copying/cloning an existing object .

Question-2: List all 7 GoF Structural patterns (Adapter, Bridge, Composite, Decorator, Facade, Flyweight, Proxy) with one-line purpose each.

Answer:

Pattern	Purpose
Adapter	Makes incompatible interfaces work together.
Bridge	Separates abstraction from implementation so both can vary independently.
Composite	Treats individual objects and groups of objects uniformly in a tree structure.
Decorator	Dynamically adds new behavior/features to an object.
Facade	Provides a simple interface to a complex subsystem.
Flyweight	Shares common objects/data to reduce memory usage.
Proxy	Provides a substitute/placeholder that controls access to another object.

Practical: Implement Singleton (thread-safe) and Adapter with short, complete Java code examples.

Answer:

1. Singleton Pattern — Thread-Safe

Purpose: Singleton ensures that a class has **only one instance** and provides a global access point.

```
class Singleton {  
  
    private static volatile Singleton instance;  
  
    private Singleton() { }
```

```

public static Singleton getInstance() {
    if (instance == null) {
        synchronized (Singleton.class) {
            if (instance == null) {
                instance = new Singleton();
            }
        }
    }
    return instance; }

public void show() {
    System.out.println("Singleton object created."); } }

public class Main {

    public static void main(String[] args) {

        Singleton s1 = Singleton.getInstance();

        Singleton s2 = Singleton.getInstance();

        s1.show();

        System.out.println(s1 == s2);

    } }

```

Output:

Singleton object created.

true

2. Adapter Pattern

Purpose: Adapter makes incompatible interfaces work together.

// Target interface

```

interface Payment {

    void pay(); }

```

// Existing class

```

class OldPayment {

```

```
void makePayment() {  
    System.out.println("Payment made.");  
} }  
  
// Adapter  
  
class PaymentAdapter implements Payment {  
    private OldPayment oldPayment;  
  
    PaymentAdapter(OldPayment oldPayment) {  
        this.oldPayment = oldPayment;  
    }  
  
    public void pay() {  
        oldPayment.makePayment();  
    } }  
  
// Main class  
  
public class Main {  
    public static void main(String[] args) {  
        OldPayment old = new OldPayment();  
        Payment payment = new PaymentAdapter(old);  
        payment.pay();  
    } }  
}
```

Output:

Payment made.